

浏览器的工作流程

当我们在浏览器地址栏输入 `www.example.com` 并按下回车时，看似简单的操作背后隐藏着一系列复杂的网络交互过程。

域名解析（DNS）

1. 本地缓存查询

- 浏览器缓存：检查是否缓存过该域名的 IP 地址。
- 系统缓存：查询操作系统 Hosts 文件
 - Windows 的 `C:\Windows\System32\drivers\etc\hosts`。
 - Linux 的 `/etc/hosts`
- 路由器缓存：检查家庭/公司路由器的 DNS 记录。

2. 递归查询

若本地无缓存，浏览器向 本地 DNS 服务器（如 `8.8.8.8`）发起请求：

1. 本地 DNS 查询 根域名服务器（Root DNS），获取顶级域（如 `.com`）的地址。
2. 向 顶级域名服务器 查询权威 DNS 服务器地址（如 `example.com` 的 NS 记录）。
3. 向 权威 DNS 服务器 获取最终 IP（如 `93.184.216.34`）。

3. 结果缓存

IP 地址被缓存在本地，TTL（Time to Live）决定有效期。

建立TCP连接

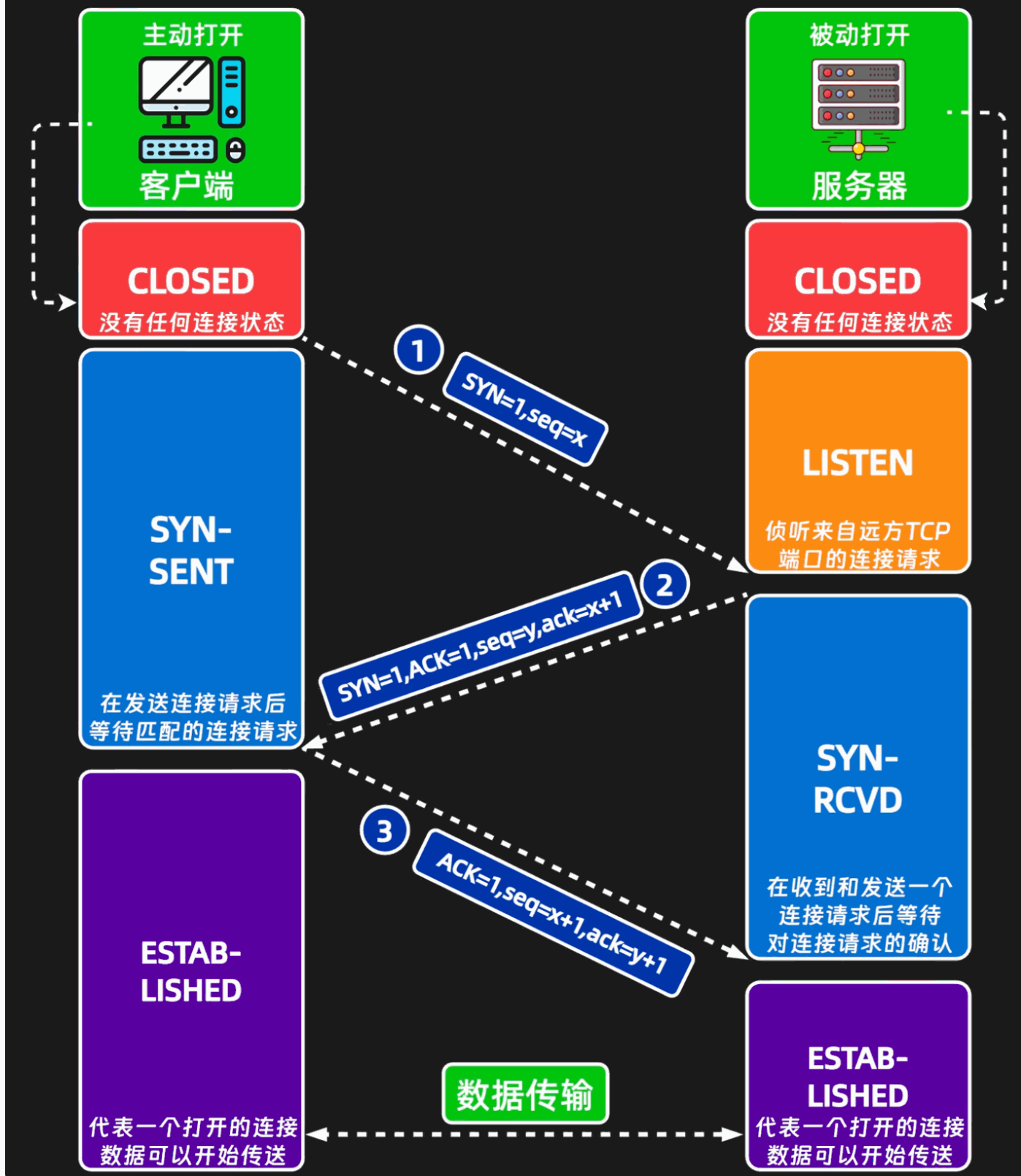
三次握手

浏览器通过 IP 地址与服务器建立 TCP 连接：

1. SYN：客户端发送同步报文。

2. **SYN-ACK**: 服务器回应同步确认。
3. **ACK**: 客户端确认。

TCP协议三次握手



TCP 三次握手流程详解

TCP 使用三次握手（Three-way Handshake）建立可靠连接，这是所有 TCP 数据传输的基础。整个过程如下：

握手流程（客户端主动发起连接）

1. 第一次握手：SYN

- 客户端 → 服务器
- 发送标志位：SYN=1
- 随机生成初始序列号：Seq=X
- 状态变化：
 - 客户端：CLOSED → SYN_SENT
- 目的：客户端向服务器发起连接请求

2. 第二次握手：SYN+ACK

- 服务器 → 客户端
- 发送标志位：SYN=1, ACK=1
- 确认号：Ack=X+1
- 随机生成初始序列号：Seq=Y
- 状态变化：
 - 服务器：LISTEN → SYN_RCVD
- 目的：服务器确认收到请求，并发出自己的连接请求

3. 第三次握手：ACK

- 客户端 → 服务器
- 发送标志位：ACK=1
- 序列号：Seq=X+1
- 确认号：Ack=Y+1
- 状态变化：
 - 客户端：SYN_SENT → ESTABLISHED
 - 服务器：SYN_RCVD → ESTABLISHED
- 目的：客户端确认服务器的响应，连接建立完成

TLS 安全握手 (HTTPS)

1. 协商加密套件

客户端发送支持的 TLS 版本和加密算法（如 `TLS_AES_256_GCM_SHA384`）。

2. 证书验证

服务器返回数字证书，浏览器验证：

- 证书链是否由可信 CA（如 Let's Encrypt）签发。
- 域名是否匹配（防止钓鱼网站）。
- 证书是否过期。

3. 密钥交换

通过 算法 生成会话密钥，后续通信使用对称加密（如 AES-GCM）。

发送 HTTP 请求

1. 请求行

```
GET /index.html HTTP/1.1
```

2. 请求头

```
Host: www.example.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
Accept: text/html,application/xhtml+xml
Cookie: session_id=abc123
```

3. 请求体

GET 请求通常无 Body，POST 请求包含表单数据或 JSON。

服务器处理请求

1. 负载均衡

请求可能被转发到集群中的某台服务器（如通过 Nginx 的 `upstream` 配置）。

2. 业务逻辑

- 静态资源：直接返回 HTML/CSS/JS 文件。
- 动态内容：通过 PHP、Node.js 或 Java 生成响应。

3. 数据库交互

查询 MySQL 或 Redis 获取数据（如用户信息）。

接收 HTTP 响应

1. 响应行

```
HTTP/1.1 200 OK
```

2. 响应头

```
Content-Type: text/html; charset=UTF-8  
Cache-Control: max-age=3600  
Set-Cookie: session_id=def456; Path=/; Secure
```

3. 响应体

```
<!DOCTYPE html>
<html>
  <head><title>Example</title></head>
  <body>Hello World!</body>
</html>
```

浏览器渲染

1. 解析 HTML

- 构建 DOM 树 (Document Object Model) 。
- 遇到 `<link>` 和 `<script>` 时阻塞渲染 (除非标记 `async/defer`) 。

2. 解析 CSS

生成 CSSOM 树 (CSS Object Model) , 与 DOM 合并为渲染树。

3. 布局与绘制

- Layout: 计算元素的位置和大小 (如 Flexbox 布局) 。
- Paint: 将渲染树转换为屏幕像素。
- Composite: 层合成优化 (如 GPU 加速) 。

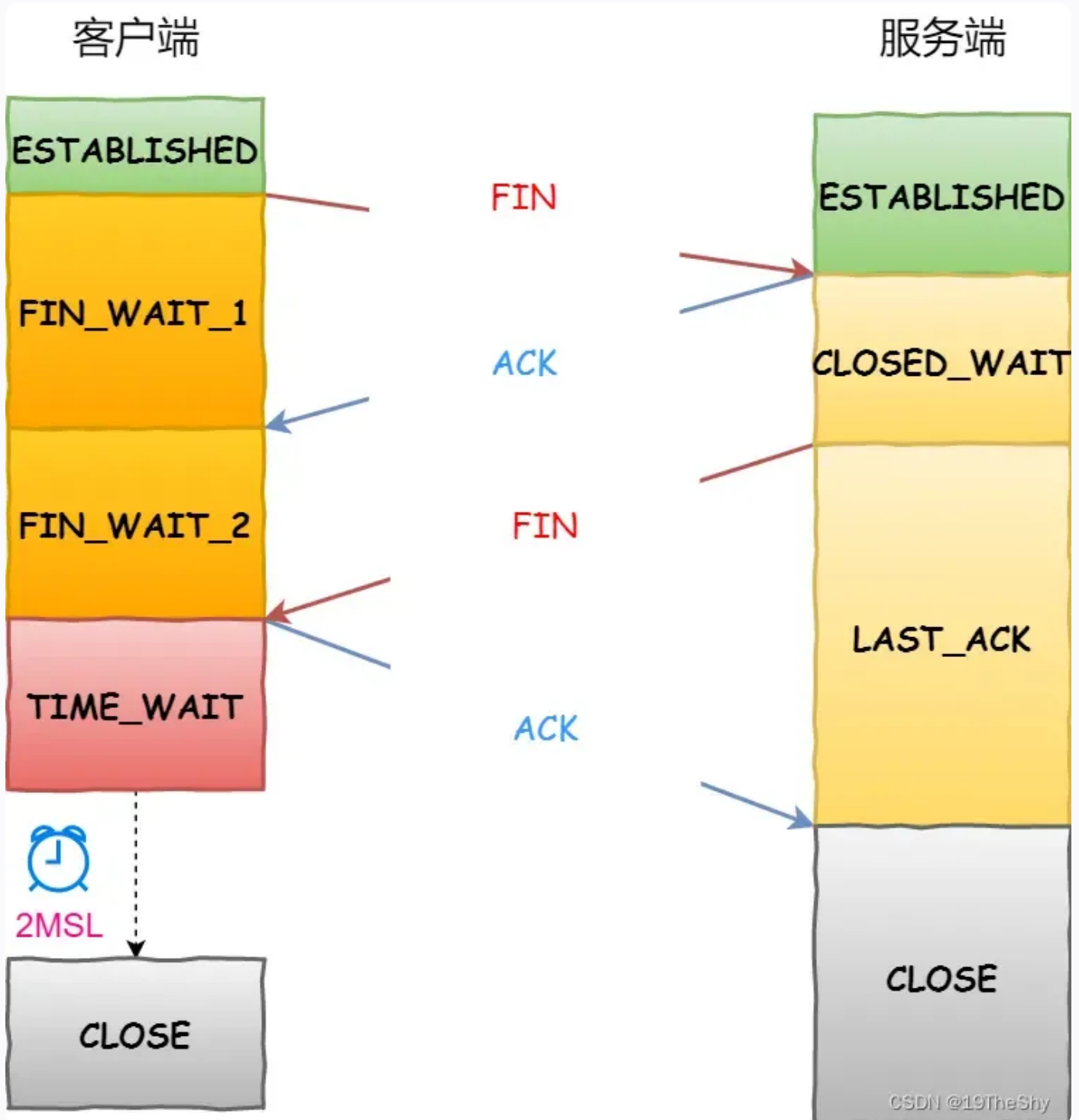
4. 执行 JavaScript

- 触发 `DOMContentLoaded` 事件 (DOM 解析完成) 。
- 所有资源加载完成后触发 `load` 事件。

连接终止

1. TCP 四次挥手

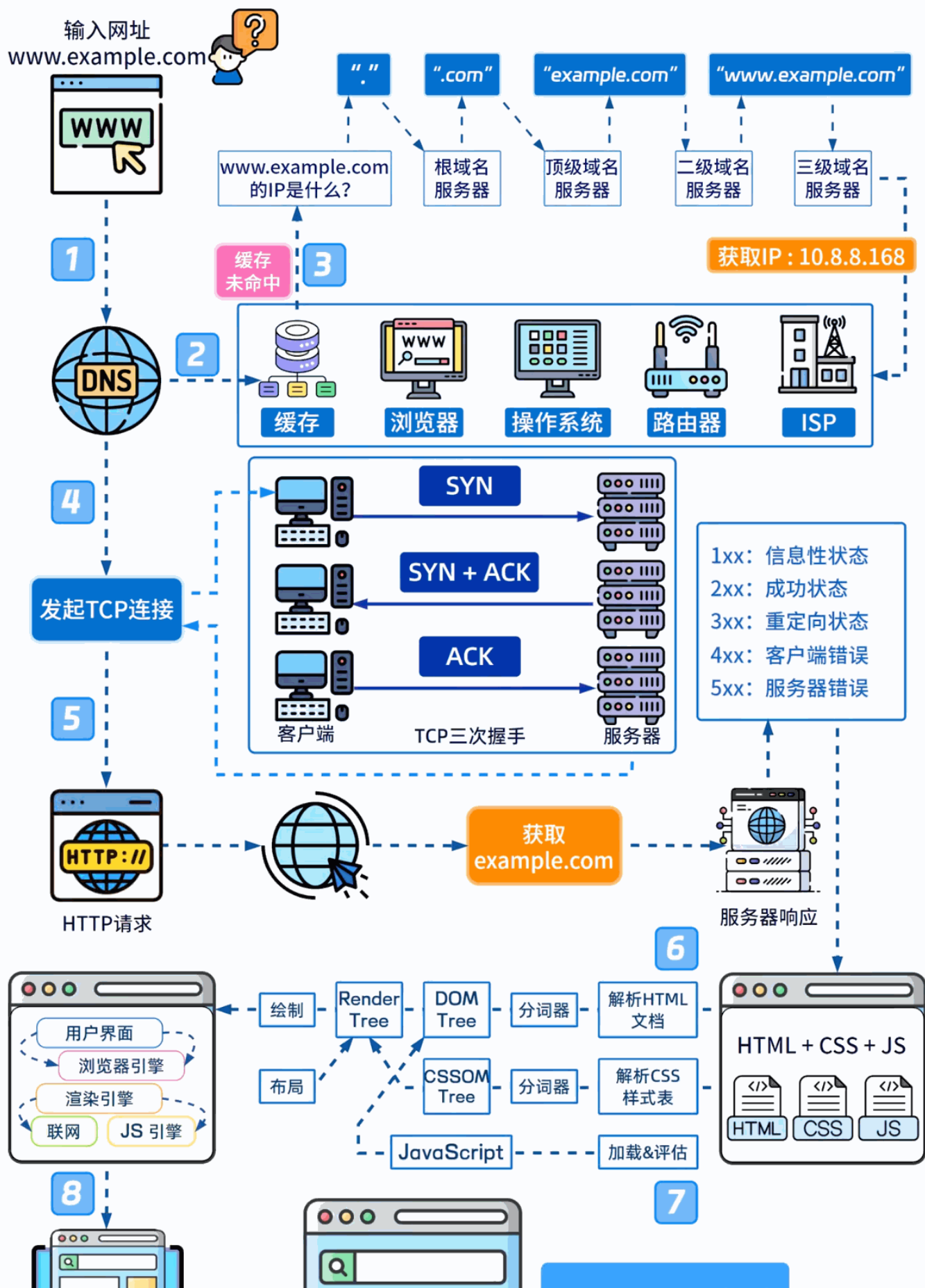
1. FIN: 客户端发送终止请求 (Seq=500) 。
2. ACK: 服务器确认 (Ack=501) 。
3. FIN: 服务器发送终止请求 (Seq=700) 。
4. ACK: 客户端确认 (Ack=701)

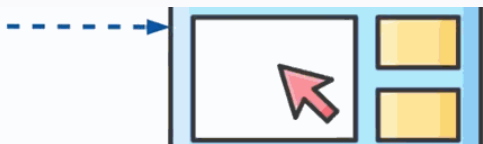


2. 持久连接

HTTP/1.1 默认启用 Keep-Alive, 复用同一 TCP 连接处理多个请求。

浏览器输入网址会发生什么





页面加载成功